



**CENTRALE
LYON**

ÉCOLE CENTRALE LYON

RAPPORT

**PAR 104 : Segmentation sémantique
vidéo pour l'analyse du développement
cognitif infantile**

Élèves :

Amel TEBBOUNE
Sarah SEBBANE

Enseignants :

Stéphane DERRODE
Zied BOUYAHIA
Juliette BLAYAC

Table des matières

1	Introduction et contexte du projet	2
1.1	Introduction	2
1.2	Contexte du projet	2
1.3	Objectifs	5
1.4	Connaissances annexes sur les bébés	6
2	État de l'art et cadrage du problème	6
2.1	Notions de pré-traitement des vidéos : débruitage et stabilisation	6
2.2	Détection d'objets – Principes et fonctionnement	9
2.3	Cadrage du problème	11
3	Implémentation et développement de l'algorithme	14
3.1	Protocole	14
3.2	Implémentation de l'algorithme de détection	14
3.3	Fine-tuning d'un modèle de Deep Learning	19
3.4	Quantification des performances de l'algorithme	24
4	Exploitation des résultats	25
4.1	Choix des vidéos et contraintes techniques	25
4.2	Choix des indicateurs	25
4.3	Visualisation des résultats	27
5	Pistes d'améliorations et conclusion	29
5.1	Pistes d'améliorations	29
5.2	Conclusion	30
6	Dépôt GitHub	30

1 Introduction et contexte du projet

1.1 Introduction

Cette étude s'inscrit dans le cadre d'un projet de recherche visant à analyser des vidéos recueillies auprès de nourrissons. L'objectif principal est d'extraire, à partir de ces enregistrements, des éléments pertinents permettant aux chercheurs en développement cognitif d'étudier la perception visuelle des nourrissons depuis leur propre perspective. Cette approche novatrice permet d'examiner directement les stimuli visuels auxquels les enfants sont exposés dans leur environnement naturel.

1.2 Contexte du projet

Commanditaire Cette recherche est menée en collaboration avec le Laboratoire de Psychologie de Bron - CNRS Babylab de Lyon, sous la direction scientifique de Jean-Rémy Hochmann. Les axes de recherche s'articulent autour de plusieurs problématiques fondamentales dans le domaine du développement cognitif. Les chercheurs s'intéressent particulièrement aux modalités d'interaction entre l'enfant et les adultes, ainsi qu'à l'influence de la posture sur les mécanismes attentionnels. L'étude porte également sur la perception des objets dans l'environnement immédiat et sur l'impact du développement moteur sur les capacités perceptives de l'enfant.

Nous avons accès à une base de données contenant des vidéos embarquées sur des bébés. Ces vidéos disposent également de quelques annotations.

Base de données Les vidéos sont classées en différentes catégories. Les informations pertinentes incluent la date de prise, l'âge et l'identification du bébé.

Les bébés sont également regroupés en catégories. Les informations présentées comprennent le nom, la date de naissance, l'ethnicité, d'éventuelles maladies (dont l'ASD pour l'un d'entre eux) ainsi que le pays d'origine, qui est majoritairement les États-Unis, sauf pour un bébé venant d'Australie.



Summarize by: folder											
type	name	test date	release	file name	ID	birthdate	age	gender	race	ethnicity	pa
materials	Misc Files Testing			2 files							
session	S_20131017_1200	2013-10-17	21	5 files	S	2012-10-17	12.0 mos	Male	White	Not Hispanic or Latino	
session	S_20131019_1202	2013-10-19	21	5 files	S	2012-10-17	12.1 mos	Male	White	Not Hispanic or Latino	
session	S_20131024_1207	2013-10-24	21	4 files	S	2012-10-17	12.2 mos	Male	White	Not Hispanic or Latino	

FIGURE 1 – Capture d'écran de la base de données

Caractéristiques des vidéos La vidéo est de mauvaise qualité, avec du bruit, des artefacts de compression et une mauvaise stabilisation. L'enfant change de position et est entouré d'objets caractéristiques de l'enfance. L'angle de prise de vue est large, ce qui arrondit les bords de

l'image.

Concernant l'utilisation du grand angle, les créateurs du corpus SayCam soutiennent qu'il permet de mieux restituer le véritable input visuel des enfants, qui peuvent aussi bouger les yeux, élargissant ainsi leur champ de vision au-delà de celui de la caméra fixe. Il peut être intéressant d'examiner les différences entre une analyse centrée sur le milieu de l'image et une prise en compte des bords. La caméra étant placée sur le front, il peut également être pertinent de privilégier la partie basse de l'image plutôt que la partie haute.

La caméra est légère, avec un objectif grand angle, une résolution de 480p et un enregistrement entre 20 et 30 images par seconde. Elle est placée sur l'enfant ou à côté de lui s'il refuse de la porter.

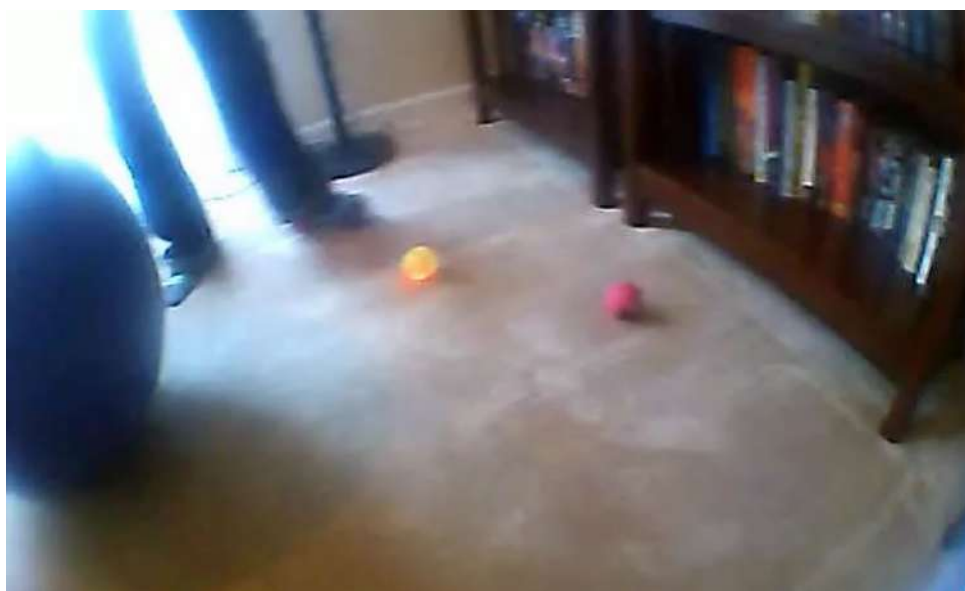


FIGURE 2 – Image tirée d'une vidéo après un premier traitement

Le corpus comprend 18 000 vidéos pour un total de 460 heures, enregistrées aux formats AVI ou MP4. Les vidéos capturent des interactions en intérieur et en extérieur, ainsi qu'avec des animaux. Une table des annotations est censée être disponible, mais en réalité, les annotations ne sont pas présentes sur les vidéos.

Locations	Activity	Living Things	Objects	Body Parts
Bathroom	Being Held	Bird(s)	Appliance	Present/Absent
Bedroom	Cleaning	Cat (with specific cat)	Bag	Head/Face
California Room	Cooking	Dad	Bed	Body
Car	Covering	Dog	Book	Arms/Hands
Closet	Crawling	Fish	Building	Legs/Feet
Deck/Porch	Crying	Grandma	Car	Other
Garage	Drawing	Grandpa	Chair	Phone*
Hallway	Drinking	Horse	Clothing	
Kitchen	Eating	Mom	Computer	
Laundry Room	Examining	Other Adult	Container	
Living Room	Exploring	Other Child	Cream	
Off Property	Gardening	Participant	Crib	
Office	Getting Changed/Dressed		Diapers/Wipes/Potty	
Outside On Property	Getting Parental Attention		Doll/Stuffed Toy	
Piano Room	Imitating		Door	
Many Locations	Listening to Music		Drawing	
Stairway	Lying Down		Drawing/Writing Implements	
	Nursing		Drink	
	Overhearing Speech		Flora	
	Painting		Food	
	Playing Music		Improvised Toy	
	Preparing for Outing		Laundry Machine	
	Reading		Linen/Cloths	
	Running		Mirror	
	Sitting		Musical Instrument	
	Standing		Outdoors	
	Taking a Walk		Phone	
	Tidying		Picture	
	Walking		Play Equipment	
	Watching		Puzzle	
			Table	
			Tool	
			Toy	
			Wagon	
			Window	

Note. In some cases, individuals were present in the dataset via phone and therefore were not embodied; for those cases, we indicated the individual's presence via "Phone" in the "Body Parts" section.

FIGURE 3 – Table des annotations

Caractéristiques des enfants Deux enfants sont issus d'une famille américaine, tandis qu'un enfant est australien. Ils sont tous enfants uniques et ont des mères psychologues. Des animaux domestiques sont présents dans leur environnement. L'un des enfants présente un léger trouble du spectre de l'autisme.



FIGURE 4 – Un des participants

Participant	Location	Age at first recording (months)	Age at last recording (months)
Sam (S)	Adelaide, Australia	6 m.	30 m.
Alice (A)	San Diego, California, USA and Saratoga Springs, New York, USA	8 m.	31 m.
Asa (Y)	Saratoga Springs, New York, USA	7 m.	24 m.

FIGURE 5 – Table des participants

1.3 Objectifs

La problématisation de ce projet repose sur la nécessité d’analyser un grand nombre de vidéos de faible qualité. L’objectif général est de développer un algorithme de segmentation sémantique capable d’identifier les objets pertinents. Cela nécessite au préalable une phase de pré-traitement pour améliorer la qualité des vidéos.

Les enjeux sont doubles : d’une part, créer un outil capable d’analyser un volume important de vidéos complexes, et d’autre part, contribuer à la recherche sur le développement cognitif.

La méthodologie du projet est structurée en trois principales phases :

- Pré-traitement : étant donné la mauvaise qualité des vidéos, il est essentiel de les débruirer et de les stabiliser avant toute analyse.
- Segmentation sémantique : cette phase vise à détecter les objets présents autour de l’enfant, analyser ses interactions et identifier sa position.
- Métriques quantitatives sur la fréquence et la distribution spatiale des objets dans le champ visuel des nourrissons, afin d’étudier l’évolution de leur attention visuelle avec l’âge.

1.4 Connaissances annexes sur les bébés

Vers 4 mois, le bébé a une vision plus claire, bien qu'il préfère toujours voir de près. Il utilise ses deux yeux (vision binoculaire) et commence à s'exercer sur la perception de la profondeur.

Entre 4 et 6 mois, il distingue mieux les reliefs, aime observer son environnement et tourne la tête pour mieux voir, notamment pour distinguer les expressions du visage. Il perçoit également mieux les teintes de base (bleu, vert, rouge) et la coordination entre ses deux yeux s'améliore.

Vers 6 mois, sa vision continue de s'affiner.

Entre 6 et 12 mois, les deux yeux travaillent ensemble, permettant à l'enfant de voir en trois dimensions. Il devient capable de repérer un petit objet situé près de lui.

Entre 7 et 12 mois, le bébé se déplace davantage, apprend à évaluer les distances et saisit mieux les objets. Il découvre également comment fonctionne sa vision lorsqu'il bouge son corps.

Vers 9 mois, les mouvements de ses yeux et de ses mains deviennent mieux coordonnés.

Entre 1 et 2 ans, la vision de loin du bébé s'améliore progressivement.

2 État de l'art et cadrage du problème

2.1 Notions de pré-traitement des vidéos : débruitage et stabilisation

Types de bruit

Bruit Gaussien Le bruit gaussien constitue une forme majeure de dégradation dans les séquences vidéo, principalement causé par l'agitation thermique des électrons et les fluctuations quantiques des photodétecteurs. Sa distribution normale, caractérisée par une moyenne μ et un écart-type σ , suit l'équation :

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (1)$$

Son impact est particulièrement marqué en conditions de faible luminosité, où il détériore uniformément l'image, affectant notamment les détails fins et les textures.

Bruit impulsif Le bruit impulsif, ou bruit "sel et poivre", se caractérise par l'apparition aléatoire de pixels isolés aux valeurs extrêmes (très clairs ou très sombres). Il résulte principalement des défauts des capteurs, des erreurs de transmission, des interférences électromagnétiques et des problèmes de conversion analogique-numérique.

Bruit de compression Le bruit de compression, inhérent aux algorithmes de compression avec perte, se manifeste par des artéfacts visuels dont l'intensité dépend du taux de compression, de la complexité de la scène et de l'amplitude du mouvement dans la séquence.

Débruitage - Méthodes classiques de débruitage

Approches spatiales Les filtres spatiaux implémentés via OpenCV offrent une solution rapide et efficace pour le débruitage vidéo. Le filtre moyennneur (`cv2.blur`), particulièrement performant en temps de calcul, opère une moyenne locale des pixels. Bien qu'il traite efficacement le bruit gaussien de faible amplitude, il peut entraîner un floutage des contours. Sa rapidité d'exécution le rend particulièrement adapté au traitement en temps réel.

Le filtre médian (`cv2.medianBlur`) remplace chaque pixel par la valeur médiane de son voisinage. Son implémentation optimisée dans OpenCV en fait un outil de choix pour le traitement rapide du bruit impulsionnel, avec une meilleure préservation des contours que le filtre moyennneur.

Méthodes spatio-temporelles L'algorithme V-BM3D étend le concept du BM3D à la dimension temporelle en exploitant la similarité des blocs entre trames successives. Cette approche, bien que plus complexe en termes de calcul que les filtres OpenCV, permet une meilleure exploitation de la redondance temporelle.

Le V-BM4D améliore ce concept en ajoutant une dimension supplémentaire au traitement des patches, offrant une meilleure gestion des mouvements complexes au prix d'un temps de calcul accru.

Débruitage - Approches par apprentissage profond

Réseaux CNN DVDnet et FastDVDnet représentent des architectures CNN multi-échelles intégrant des connexions résiduelles. FastDVDnet se distingue par son traitement en temps réel, bien que restant plus lent que les filtres OpenCV classiques.

Architectures récurrentes Le RSDN (Recurrent Scene Denoising Network) utilise des cellules LSTM adaptées au traitement vidéo, permettant une propagation efficace de l'information temporelle.

Transformers Le VRT (Video Restoration Transformer) représente l'état de l'art actuel, utilisant un mécanisme d'attention multi-tête spatio-temporelle.

Débruitage - Analyse comparative des approches Les filtres OpenCV offrent un excellent compromis entre rapidité d'exécution et qualité de débruitage pour de nombreuses applications temps réel. Leur implémentation optimisée permet un traitement vidéo fluide avec une consommation mémoire minimale. Les méthodes d'apprentissage profond, bien que plus performantes en termes de PSNR, requièrent des ressources computationnelles significativement plus importantes. Le choix entre ces approches dépend donc principalement des contraintes temporelles de l'application : pour du traitement en temps réel avec des ressources limitées, les filtres OpenCV constituent la solution privilégiée, tandis que les approches par apprentissage profond sont préférables lorsque la qualité prime sur la vitesse d'exécution.

Stabilisation La stabilisation vidéo vise à éliminer les mouvements indésirables de la caméra pour produire des séquences plus fluides. Ce processus s'articule autour de trois étapes fondamentales : l'estimation du mouvement, son lissage pour obtenir une trajectoire stable, et la composition finale de l'image. Ces étapes forment un pipeline complet de stabilisation dont l'efficacité détermine la qualité du résultat final.

Stabilisation - Processus détaillé

Estimation du mouvement L'estimation du mouvement représente la première étape cruciale du processus de stabilisation. Cette phase vise à déterminer avec précision les transformations géométriques entre les frames consécutives de la séquence vidéo. Plusieurs approches techniques permettent d'accomplir cette tâche.

Le flux optique (Optical Flow) constitue une méthode fondamentale, mesurant les déplacements des pixels entre deux images successives. Cette technique permet une analyse fine des mouvements locaux dans l'image. La méthode Good Features to Track, implémentée dans OpenCV, enrichit cette approche en identifiant des points d'intérêt particulièrement pertinents pour le suivi. L'algorithme Lucas-Kanade Optical Flow, quant à lui, se concentre sur l'estimation des mouvements locaux avec une précision accrue.

Ces mouvements peuvent être modélisés selon différentes transformations géométriques. La transformation affine permet de capturer les translations, rotations et changements d'échelle. La transformation euclidienne, plus restrictive, se limite aux translations et rotations. L'homographie, plus complexe, peut représenter des déformations perspectives, particulièrement utiles pour les mouvements de caméra sophistiqués.

Lissage du mouvement Le lissage du mouvement constitue une étape déterminante pour obtenir une trajectoire stable. Cette phase vise à filtrer les variations rapides et indésirables dans le mouvement estimé, tout en préservant les mouvements intentionnels de la caméra. Plusieurs techniques de lissage peuvent être employées. La moyenne mobile (sliding window) offre une approche simple mais efficace, calculant pour chaque frame une moyenne des transformations sur une fenêtre temporelle donnée. Le filtrage par convolution, utilisant notamment des noyaux gaussiens, permet un lissage plus sophistiqué adapté aux trajectoires complexes. Le lissage cumulatif suivi de lissage représente une approche plus avancée, particulièrement efficace pour les mouvements de grande amplitude.

Composition de l'image La composition de l'image représente l'étape finale du processus de stabilisation, où les transformations calculées sont appliquées pour produire la séquence stabilisée. Cette phase nécessite une attention particulière pour maintenir la qualité visuelle tout en compensant les mouvements indésirables. Les transformations lissées sont appliquées via des fonctions spécialisées comme WarpAffine ou WarpPerspective d'OpenCV. Ces opérations nécessitent souvent des ajustements aux bordures de l'image, pouvant inclure l'ajout de bandes noires ou l'application d'un léger zoom pour compenser les zones manquantes. La gestion de ces bordures constitue un aspect crucial pour la qualité du résultat final.

Stabilisation - Implémentations et solutions techniques

VidStab Python VidStab représente une solution accessible et efficace pour la stabilisation vidéo. Cette bibliothèque, construite sur OpenCV, implémente une approche par fenêtres de lissage pour stabiliser le mouvement global.

Approche OpenCV manuelle L'approche manuelle utilisant OpenCV offre un contrôle plus fin sur le processus de stabilisation. Cette méthode implique une séquence d'opérations précises : la détection de points clés via `goodFeaturesToTrack`, le calcul du flux optique avec `calcOpticalFlowPyrLK`, l'estimation des transformations géométriques par `estimateAffine2D`, et l'application des transformations via `warpAffine`. Cette approche, bien que plus complexe, permet une personnalisation poussée adaptée aux besoins spécifiques.

Solutions basées sur l'apprentissage profond Les techniques de Machine Learning et Deep Learning représentent l'état de l'art en matière de stabilisation vidéo. Les réseaux neuronaux peuvent estimer et compenser les mouvements de manière plus sophistiquée que les approches traditionnelles. Les architectures GAN (Generative Adversarial Networks) permettent une régularisation innovante des frames consécutives, produisant des résultats particulièrement fluides et naturels.

2.2 Détection d'objets – Principes et fonctionnement

La détection d'objets dans les vidéos est une branche de la vision par ordinateur. Son but est d'identifier et de localiser des objets dans chaque image d'une vidéo. Cette tâche est souvent contrainte par la nécessité d'un traitement en temps réel.

Détection d'objets – Les bases avec les réseaux de neurones convolutifs (CNN)

Extraction des caractéristiques Les réseaux de neurones convolutifs (CNN) sont essentiels pour la détection d'objets. Ils analysent les images en appliquant des filtres pour repérer des formes et des motifs, allant des éléments simples comme les contours aux structures plus complexes.

Sélection et réduction Après l'extraction, une phase de sélection permet de ne conserver que les informations les plus importantes. Des techniques comme le *pooling* réduisent la taille des données tout en gardant les éléments essentiels, ce qui allège les calculs et rend le modèle plus robuste aux petites variations dans les images.

Prédiction finale Enfin, les dernières couches du réseau attribuent une probabilité à chaque objet détecté et déterminent ses coordonnées dans l'image.

Détection d'objets – YOLO

Une approche unifiée YOLO (*You Only Look Once*) détecte et classe les objets en une seule étape, contrairement aux méthodes classiques qui séparaient ces tâches. L'image est divisée en une grille, et chaque cellule prédit directement la présence d'un objet, sa position, sa taille et sa catégorie.

Exploitation des images successives Dans une vidéo, le suivi des objets entre les images permet d'améliorer la détection. Des algorithmes exploitent la continuité temporelle pour suivre les objets, éviter les erreurs de détection et limiter les calculs inutiles. Cela permet aussi d'analyser leurs mouvements et de gérer les moments où ils sont temporairement cachés.

Modèle	FPS	AP (%)	Taille
YOLOv3	35	31,0	416×416
YOLOv4	54	41,2	416×416
Faster R-CNN	9,4	39,8	Variable
SSD	43	25,1	300×300
EfficientDet-D0	62,5	33,8	512×512

TABLE 1 – Comparaison des performances des modèles de détection d'objets sur MS COCO

Performance - objets Par FPS, on entend Frame Per Seconde et AP, la précision moyenne (Average Precision). COCO (Common Objects in Context) désigne un jeu de données largement utilisé en vision par ordinateur, notamment pour l'entraînement et l'évaluation des modèles de détection, segmentation et reconnaissance d'objets.

YOLOv4 a été retenu pour notre projet en raison de ses performances supérieures : il atteint 54 FPS avec une précision de 41,2% sur COCO, surpassant significativement YOLOv3 tout en maintenant une exécution temps réel. Son architecture optimisée pour GPU grand public en fait une solution idéale pour un déploiement à grande échelle avec un excellent rapport coût-performance.

Modèle	FPS	Performances	Environnement
MTCNN	20	94.4% sur FDDB	CPU
RetinaFace	26.5	91.8% AP sur WIDER FACE	GPU (Tesla V100)
BlazeFace	>500	AP 0.996	Mobile GPU

TABLE 2 – Comparaison des modèles de détection de visages

Performances - visages MTCNN a été retenu pour notre projet car il offre un bon équilibre entre vitesse (20 FPS sur CPU) et précision (94.4% sur FDDB). Sa performance éprouvée et sa capacité à fonctionner efficacement sur CPU standard en font une solution fiable pour notre traitement vidéo, même si BlazeFace propose une vitesse supérieure sur GPU mobile.

Framework	FPS	Applications	Plateforme
MediaPipe	30+	Multi-usage	Mobile/Desktop
OpenPose	22-32	Pose estimation	GPU (CUDA)
Detectron2	26*	Détection générique	GPU (PyTorch)

TABLE 3 – Comparaison des frameworks de vision par ordinateur

Performances - mains (poses) Pour le traitement en temps réel des images, nous avons sélectionné MediaPipe car il offre une performance optimisée de 30+ FPS sur des plateformes

mobiles et desktop, sans nécessiter de GPU dédié. Bien qu'OpenPose offre des capacités avancées d'estimation de pose sur GPU (22-32 FPS) et que Detectron2 propose une architecture modulaire puissante basée sur PyTorch, MediaPipe représente la solution la plus adaptée à notre besoin de traitement rapide et déployable sur différentes plateformes.

2.3 Cadrage du problème

Classification des objets Jusqu'à l'âge de 10 mois, les enfants distinguent principalement les personnes et les animaux des objets inanimés. Notre modélisation s'appuie sur les catégories lexicales émergentes, où l'acquisition d'un mot témoigne généralement de la maîtrise de la catégorie sous-jacente. La classification suivante s'appuie sur les recommandations de JRH.

L'analyse des stimuli visuels infantiles révèle les distinctions suivantes :

- **Personnes**
- **Animaux**
- **Objets naturels :**
 - Grands (arbres, végétation)
 - Petits (fruits/fleurs, rarement saisis)
- **Objets artificiels :**
 - Grands (meubles, véhicules)
 - Petits (jouets, ustensiles)

La nature spécifique de notre base d'images vidéo nécessite plusieurs adaptations méthodologiques. Tout d'abord, nous conservons de manière distincte les visages et les mains en raison de leur importance cruciale dans les interactions précoces, bien que cette séparation s'écarte de notre classification initiale regroupant ces éléments sous la catégorie "personne". Ensuite, nous fusionnons les petits objets naturels et artificiels en une seule catégorie, cette décision étant motivée par la faible occurrence des objets naturels dans le corpus et leur similarité fonctionnelle dans les interactions infantiles.

La schématisation finale comprend six classes opérationnelles :

visages

mains

A Animaux

OAG Objets Artificiels Grands

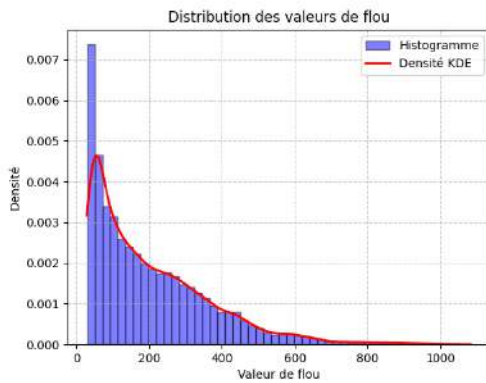
ONG Objets Naturels Grands

OP Objets Petits (fusion naturels/artificiels)

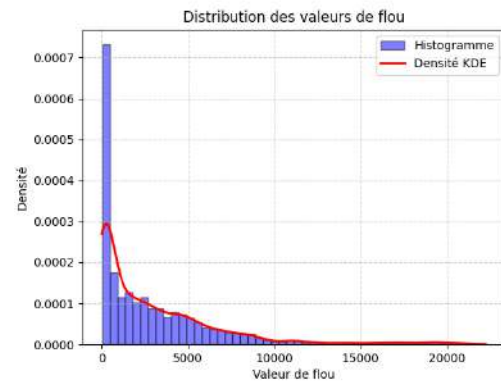
Analyse du bruit Les vidéos utilisées dans notre étude présentent un niveau de bruit significatif, en raison des limitations du matériel d'enregistrement et de leur ancienneté. Toutefois, l'analyse statistique révèle que les valeurs de bruit suivent des distributions similaires d'une vidéo à l'autre. Afin d'évaluer quantitativement ce bruit, nous utilisons la variance du Laplacien, définie comme :

$$\sigma^2 = \frac{1}{N} \sum_{i=1}^N (L_i - \mu_L)^2$$

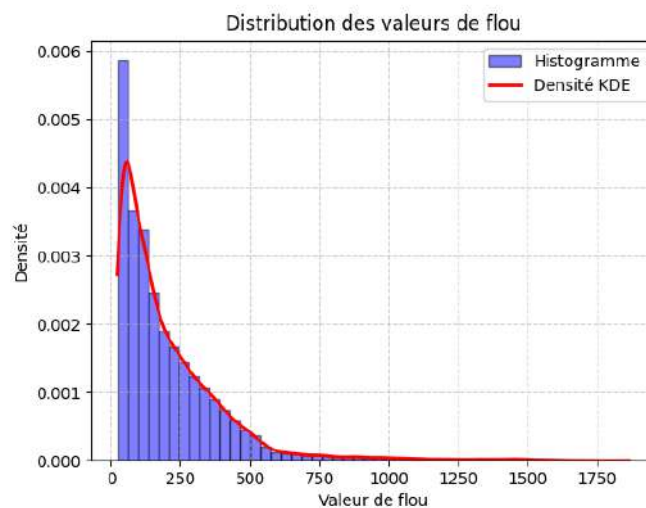
où L_i représente la valeur du Laplacien au pixel i , et μ_L est la moyenne du Laplacien sur l'image. Cette mesure nous permet d'établir un seuil de bruit acceptable, au-delà duquel les vidéos nécessitent un prétraitement pour améliorer la qualité des analyses effectuées. En deçà, les frames sont considérées inutilisables.



(a) Figure 1



(b) Figure 2



L'analyse des distributions révèle que chaque vidéo présente un profil caractéristique de forme semi-lorentzienne, où les images présentant le flou le plus prononcé se situent systématiquement dans la partie gauche de l'histogramme. Pour la segmentation des données, nous avons établi un seuil critique $t_r = 50$ permettant de différencier les frames acceptables (au-dessus du seuil) des frames trop dégradées nécessitant un traitement particulier (en-dessous du seuil). Cette approche quantitative, basée sur un critère objectif déterminé empiriquement, offre une méthode robuste pour évaluer la qualité visuelle des séquences tout en préservant l'information essentielle.

Optimisation du nombre d'images Après l'élimination des images inutilisables, une réduction supplémentaire du volume de données peut être réalisée. Notre approche repose sur l'observation qu'entre des frames successives, les détections présentent généralement une forte redondance avec peu de nouvelles occurrences significatives.

Cette propriété nous permet d'appliquer un échantillonnage temporel efficace, par exemple en conservant uniquement une frame sur cinq. Ce facteur d'échantillonnage représente un compromis optimal entre, d'une part, la réduction du temps de calcul et des redondances, et d'autre part, le maintien d'une couverture temporelle suffisante pour l'analyse.

La valeur précise du taux d'échantillonnage peut être ajustée dynamiquement en fonction de plusieurs paramètres : la fréquence d'apparition des nouveaux objets dans la scène, la vitesse de déplacement des éléments détectés, et les exigences spécifiques de précision temporelle requises par l'application. Cette méthode permet ainsi d'optimiser significativement l'efficacité du traitement tout en préservant l'intégrité des informations pertinentes pour les tâches de détection.

3 Implémentation et développement de l'algorithme

3.1 Protocole

L'analyse vidéo s'effectue selon une méthodologie en quatre phases successives. La première étape consiste en un prétraitement comprenant un débruitage du signal visuel pour éliminer les artefacts, suivi d'une stabilisation des séquences afin de corriger les mouvements parasites. La seconde phase met en œuvre des algorithmes de détection pour identifier et localiser les différentes classes d'objets présentes dans la vidéo. Les résultats sont ensuite structurés sous forme de données tabulaires exportées dans un fichier CSV. Enfin, la dernière étape permet la visualisation et l'interprétation des résultats obtenus.

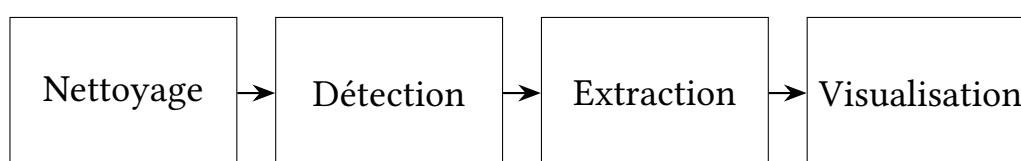


Schéma du traitement

Système de classification La schématisation finale comprend six classes opérationnelles :

visages

mains

A Animaux

OAG Objets Artificiels Grands

ONG Objets Naturels Grands

OP Objets Petits (fusion naturels/artificiels)

3.2 Implémentation de l'algorithme de détection

Introduction Cette section présente l'implémentation d'un système avancé de détection et de suivi d'objets pour l'analyse vidéo. Notre approche se distingue des méthodes traditionnelles par l'intégration de plusieurs techniques d'optimisation, notamment le traitement par lots, le suivi d'objets (tracking), et la détection intelligente de flou. Ces améliorations permettent non seulement d'accroître la précision des détections, mais aussi de réduire considérablement le temps de traitement, un facteur crucial pour l'analyse de vidéos volumineuses.

Premier algorithme développé Notre démarche initiale a consisté à développer un algorithme de base pour la détection et le traitement des objets dans les vidéos. Cette première approche, bien que fonctionnelle, présentait certaines limitations qui ont motivé nos améliorations ultérieures.

Structure de l'algorithme initial L'algorithme initial repose sur une architecture simple mais robuste, avec trois composants principaux :

- **VideoProcessor** : Responsable de l'extraction des métadonnées de la vidéo (dimensions, nombre de frames).
- **YOLOClassMapper** : Gère les correspondances entre les étiquettes YOLO en anglais et nos catégories personnalisées.
- **DetectionManager** : Coordonne le processus de détection frame par frame.

Le processus de détection suit une approche séquentielle classique :

1. Chargement de la vidéo frame par frame
2. Application des détecteurs (YOLO, MTCNN pour les visages, MediaPipe pour les mains) sur chaque frame
3. Agrégation des résultats et calcul des statistiques
4. Génération de cartes de chaleur et exportation des résultats

Chaque détection est représentée par deux structures de données principales :

- **DetectionFrame** : Stocke les informations de détection pour une frame spécifique (objets détectés, scores de confiance)
- **DetectionClass** : Agrège les informations de détection pour une catégorie d'objets sur l'ensemble de la vidéo

Fonctionnement du traitement L'algorithme original traite chaque frame indépendamment dans une boucle séquentielle :

Pour chaque frame $f \in 1, 2, \dots, N$:

1. Charger la frame
2. Appliquer les détecteurs de visages (MTCNN)
3. Appliquer les détecteurs de mains (MediaPipe)
4. Appliquer le détecteur d'objets général (YOLO)
5. Stocker les résultats

Le score final pour chaque classe de détection est calculé selon la formule :

$$\text{Score} = \frac{\text{Nombre total de détections}}{\text{Nombre de frames}} \times \text{Couverture spatiale} \times \text{Confiance moyenne} \quad (3)$$

où la couverture spatiale représente la proportion de l'image couverte par les détections de cette classe.

Limitations de l'approche initiale Cette approche initiale présentait plusieurs limitations significatives. Tout d'abord, le traitement séquentiel image par image n'exploite pas les capacités de parallélisation offertes par les GPU modernes, ce qui limite fortement les performances. Ensuite, l'absence de continuité temporelle, due au traitement indépendant de chaque frame, empêche le suivi des objets d'une image à l'autre. De plus, le modèle applique le même traitement à toutes les frames, sans distinction de qualité, ce qui le rend sensible au bruit et au flou. On observe également une redondance importante des calculs, les mêmes objets étant détectés à plusieurs reprises sans mutualisation de l'information entre les frames. Enfin, cette méthode implique un temps de traitement particulièrement élevé, pouvant atteindre plusieurs heures pour une vidéo de quelques minutes.

Ces limitations ont motivé le développement d'une approche plus sophistiquée intégrant le traitement par lots, le suivi d'objets et la détection de flou, que nous décrivons dans les sections suivantes.

Méthodologie

Architecture globale Notre système repose sur une architecture modulaire composée de plusieurs composants spécialisés : un détecteur d'objets multi-classes (YOLO), un détecteur de visages (MTCNN), un détecteur de mains (MediaPipe), un système de suivi d'objets (DeepSort), et un module de détection et de traitement du flou. La Figure 7 illustre cette architecture.

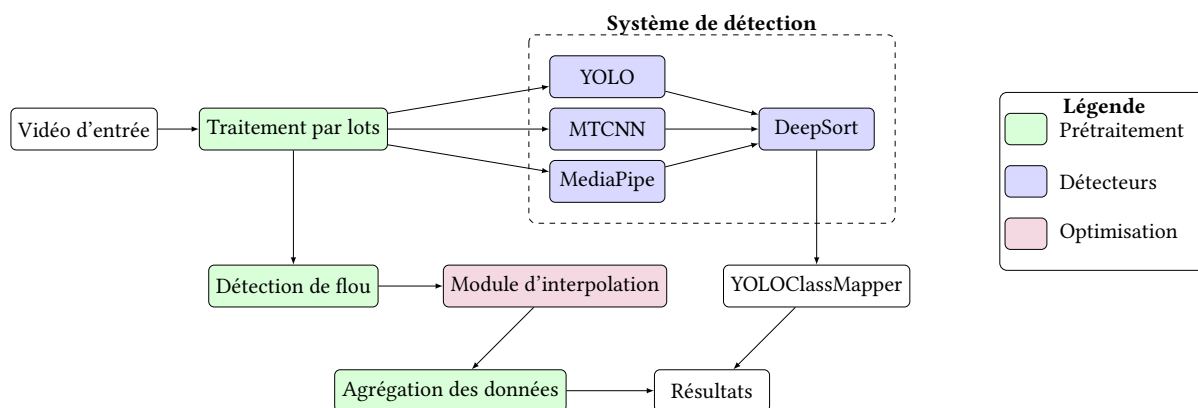


FIGURE 7 – Architecture du système d'analyse vidéo proposé

Traitement par lots Contrairement aux approches traditionnelles qui traitent les frames individuellement, notre implémentation repose sur un traitement par lots (batch processing). Cette méthode offre plusieurs avantages. Elle permet tout d'abord une parallélisation des calculs, en traitant simultanément plusieurs frames, ce qui optimise l'utilisation des capacités parallèles des GPU modernes. De plus, elle réduit les surcoûts liés à l'initialisation des modèles, comme YOLO et MTCNN, qui bénéficient d'un chargement unique pour traiter plusieurs frames à la fois. Enfin, la localité temporelle des données dans les vidéos améliore l'efficacité du CPU et du GPU en optimisant la gestion de la mémoire cache.

L'équation suivante modélise le gain de performance obtenu par le traitement par lots :

$$T_{batch} \approx \frac{T_{single} \times N}{K + N} \quad (4)$$

où T_{batch} est le temps de traitement par lots, T_{single} le temps de traitement d'une seule frame, N le nombre de frames, et K un coefficient lié aux surcoûts d'initialisation. Dans nos expériences, avec une taille de lot de 8, nous avons observé des gains de performance d'environ 65% par rapport au traitement frame par frame.

Système de suivi multi-objets L'intégration de DeepSort comme système de suivi (tracking) constitue une amélioration majeure par rapport aux approches de détection simples. DeepSort combine des features visuelles et un filtre de Kalman pour suivre les objets entre les frames successives. Son fonctionnement peut être résumé par l'équation suivante :

$$\mathbf{x}_k = \mathbf{F}_k \mathbf{x}_{k-1} + \mathbf{B}_k \mathbf{u}_k + \mathbf{w}_k \quad (5)$$

où $\mathbf{x}_k$ est l'état de l'objet à l'instant k , $\mathbf{F}_k$ la matrice de transition d'état, $\mathbf{B}_k$ la matrice de contrôle, $\mathbf{u}_k$ le vecteur de contrôle, et $\mathbf{w}_k$ le bruit du processus.

Les avantages de cette approche sont multiples. Tout d'abord, elle permet de maintenir l'identité des objets, chaque objet détecté recevant un identifiant unique qui persiste d'une frame à l'autre. De plus, la robustesse aux occlusions temporaires est assurée grâce au filtre de Kalman, qui permet de prédire la position des objets même lorsqu'ils sont temporairement masqués. L'approche permet également une réduction des faux positifs, car le suivi temporel permet de filtrer les détections isolées et incohérentes. Enfin, la précision spatiale est améliorée, car la fusion des détections sur plusieurs frames permet de mieux localiser les objets.

Détection et gestion du flou Notre implémentation intègre un système innovant de détection de flou basé sur la variance du Laplacien, définie par :

$$V = \text{Var}(\nabla^2 I) \quad (6)$$

où $\nabla^2 I$ est le Laplacien de l'image I . Cette mesure permet d'identifier automatiquement les frames floues qui contiennent moins d'informations exploitables.

Deux stratégies d'optimisation sont appliquées pour améliorer la détection. La première consiste en un calcul adaptatif du seuil de flou, où le système détermine automatiquement le seuil optimal pour la détection du flou en analysant un échantillon représentatif de la vidéo. La deuxième stratégie repose sur l'interpolation des détections. Plutôt que de traiter directement les frames floues, souvent nombreuses dans certaines vidéos, le système interpole les détections à partir des frames nettes adjacentes, ce qui permet de maintenir la qualité des résultats tout en réduisant le traitement des frames de mauvaise qualité.

La méthode d'interpolation peut être formalisée comme suit :

$$D_i = \alpha D_{i-1} + (1 - \alpha) D_{i+1} \quad (7)$$

où D_i représente les détections à la frame i (floue), D_{i-1} et D_{i+1} les détections aux frames nettes adjacentes, et α un coefficient de pondération inversement proportionnel à la distance.

Discussion

Avantages de l'approche proposée Notre implémentation présente plusieurs avantages significatifs par rapport aux approches traditionnelles. Tout d'abord, elle offre une robustesse accrue, maintenant des performances élevées même dans des conditions difficiles telles que les mouvements rapides, les occlusions ou les variations d'éclairage. De plus, son architecture modulaire permet une grande scalabilité, facilitant l'ajout de nouveaux détecteurs ou le remplacement de composants existants. En termes d'efficacité computationnelle, les optimisations mises en place permettent de traiter des vidéos de plus grande durée tout en utilisant des ressources matérielles limitées. Enfin, le système intègre un filtrage intelligent, en détectant les frames floues et en les ignorant, ce qui permet de concentrer les ressources de calcul sur les segments vidéo les plus informatifs.

Limitations et perspectives Malgré ses performances prometteuses, notre système présente certaines limitations. Tout d'abord, il reste dépendant des modèles pré-entraînés, ce qui signifie que ses performances sont étroitement liées à la qualité des modèles de détection utilisés. De plus, bien que le traitement par lots offre des avantages en termes de parallélisation, il nécessite une plus grande consommation de mémoire GPU par rapport aux approches séquentielles. Enfin, l'interpolation des détections, bien qu'efficace dans de nombreux cas, peut introduire des erreurs, particulièrement dans les scènes à mouvement rapide, ce qui limite sa précision dans certains contextes.

Les perspectives d'amélioration de notre système incluent plusieurs axes de développement. Premièrement, l'intégration de modèles d'attention temporelle pourrait améliorer la cohérence des détections en prenant mieux en compte les relations temporelles entre les frames. Ensuite, l'implémentation d'une stratégie adaptative pour la taille des lots, qui s'ajusterait en fonction de la complexité de la scène, permettrait d'optimiser davantage l'utilisation des ressources tout en maintenant des performances élevées.

3.3 Fine-tuning d'un modèle de Deep Learning

Annotation des données Le processus d'adaptation du modèle a commencé par l'annotation d'un corpus initial de 300 images, ensuite augmenté par des techniques de data augmentation pour atteindre un jeu de données trois fois plus important. L'implémentation a été réalisée avec les frameworks Torch et Ultralytics, exploitant l'architecture YOLOv8. Les premiers résultats ont révélé que le modèle pré-entraîné sur COCO démontrait des performances satisfaisantes pour la plupart des classes, à l'exception notable des Objets Petits (OP). Cette observation nous a conduits à adopter une approche duale : utiliser le modèle COCO pour les classes standards tout en développant un modèle spécialisé pour les OP.

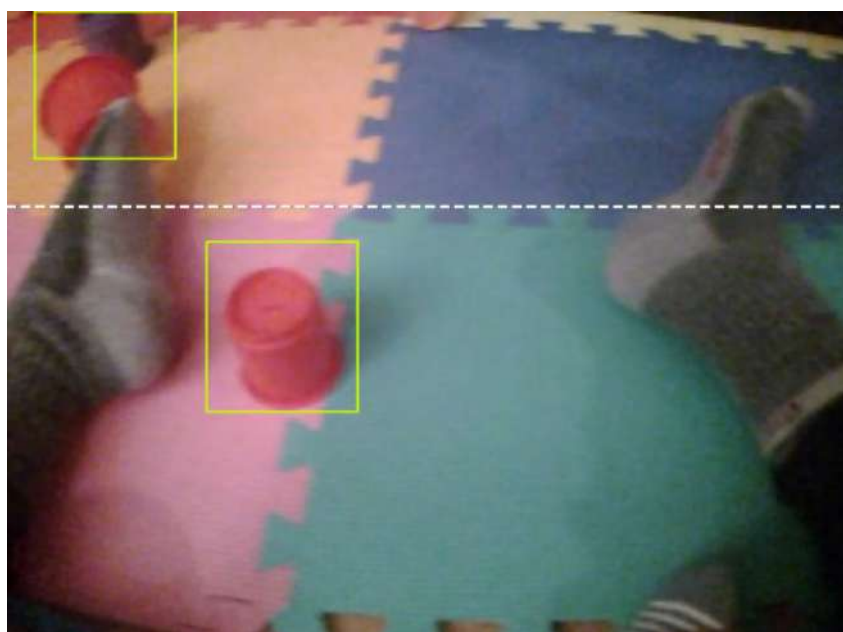


FIGURE 8 – L'annotation a été réalisée à l'aide de Roboflow.

Approche spécialisée pour les Objets Petits (OP) Le choix de YOLO s'est avéré pertinent pour sa simplicité d'utilisation et son efficacité en détection d'objets, notamment grâce à sa rapidité d'inférence et sa facilité d'adaptation à de nouveaux jeux de données.

Pour optimiser le processus d'annotation, une approche itérative a été adoptée avec six cycles successifs d'annotation manuelle (totalisant 900 images). Le premier entraînement a utilisé 50 époques, un batch size de 16 et une résolution d'image de 640 pixels. Le modèle initial, composé de 72 couches et environ 3 millions de paramètres, a fourni des résultats modestes avec une précision de 0.615 et un rappel de 0.554. Un second entraînement sur l'ensemble du corpus annoté, avec des paramètres similaires mais un taux d'apprentissage réduit pour éviter le sur-apprentissage, a augmenté la complexité du modèle à 11 millions de paramètres sans amélioration significative des performances (précision 0.595, rappel 0.614). Cette stagnation s'explique par une spécialisation excessive du modèle sur les premières annotations. L'ajout de transformations et un nouvel ajustement des hyperparamètres ont finalement permis d'atteindre une précision de 0.662 et un rappel de 0.641.

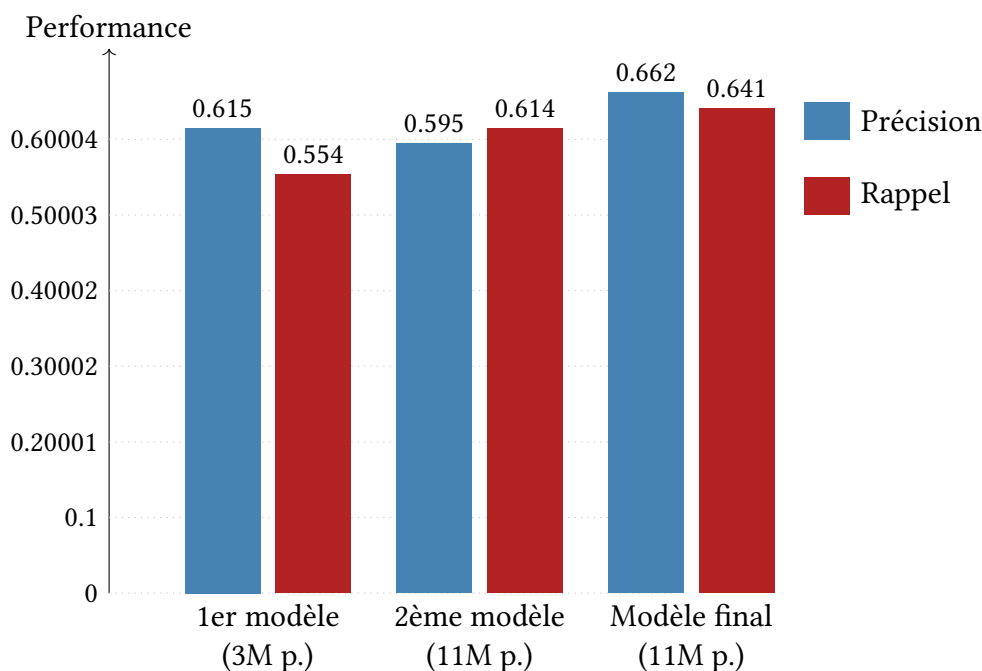


FIGURE 9 – Évolution des performances (précision et rappel) du modèle de détection des OP

Analyse de l'entraînement et des performance du modèle final L'entraînement du modèle YOLOv8 spécialisé pour la détection des objets petits (OP) a été effectué sur 100 époques, avec une désactivation de l'augmentation Mosaic à partir de l'époque 90. Les Figures 10, 11 et 12 présentent l'évolution des différentes métriques au cours de cet entraînement.

La Figure 10 montre une diminution régulière des trois fonctions de perte (Box Loss, Class Loss et DFL Loss). La Class Loss, initialement la plus élevée (2.958), connaît la réduction la plus importante pour atteindre 1.531. Cette diminution s'accélère après la désactivation de Mosaic à l'époque 90, suggérant que cette technique d'augmentation, bien qu'utile pour la généralisation, peut limiter la précision de classification en fin d'entraînement. La Box Loss (de 2.023 à 1.339) et la DFL Loss (de 2.239 à 1.855) suivent des trajectoires plus régulières, indiquant que la localisation des objets a représenté un défi moins variable que leur classification.

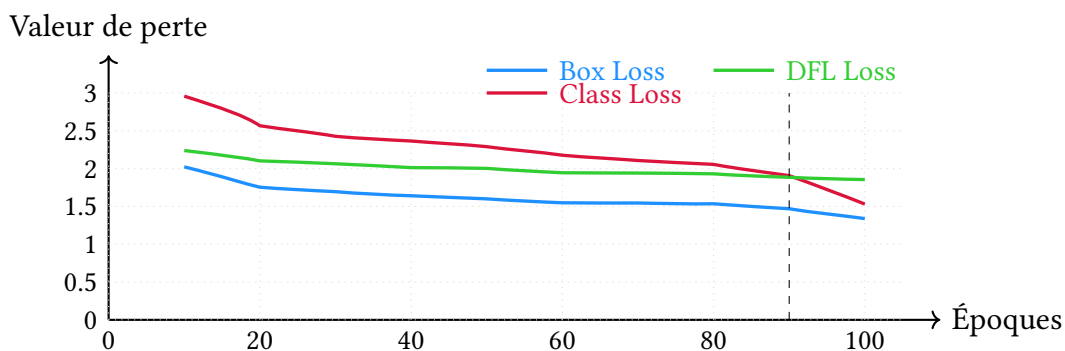


FIGURE 10 – Évolution des fonctions de perte pendant l'entraînement du modèle YOLOv8. La ligne en pointillés indique la désactivation de l'augmentation Mosaic à l'époque 90.

La Figure 11 illustre des progressions différentes pour la précision et le rappel. La précision augmente de façon substantielle, passant de 0.233 à 0.708, avec une croissance particulièrement marquée entre les époques 60 et 70. Le rappel évolue de manière plus irrégulière, de 0.209 à 0.528, avec plusieurs plateaux et même une légère diminution dans les dernières époques. L'écart final entre ces deux métriques (0.708 vs 0.528) révèle un modèle privilégiant la fiabilité des détections au détriment de l'exhaustivité – une caractéristique souvent observée dans les modèles de détection d'objets petits et difficiles à identifier.

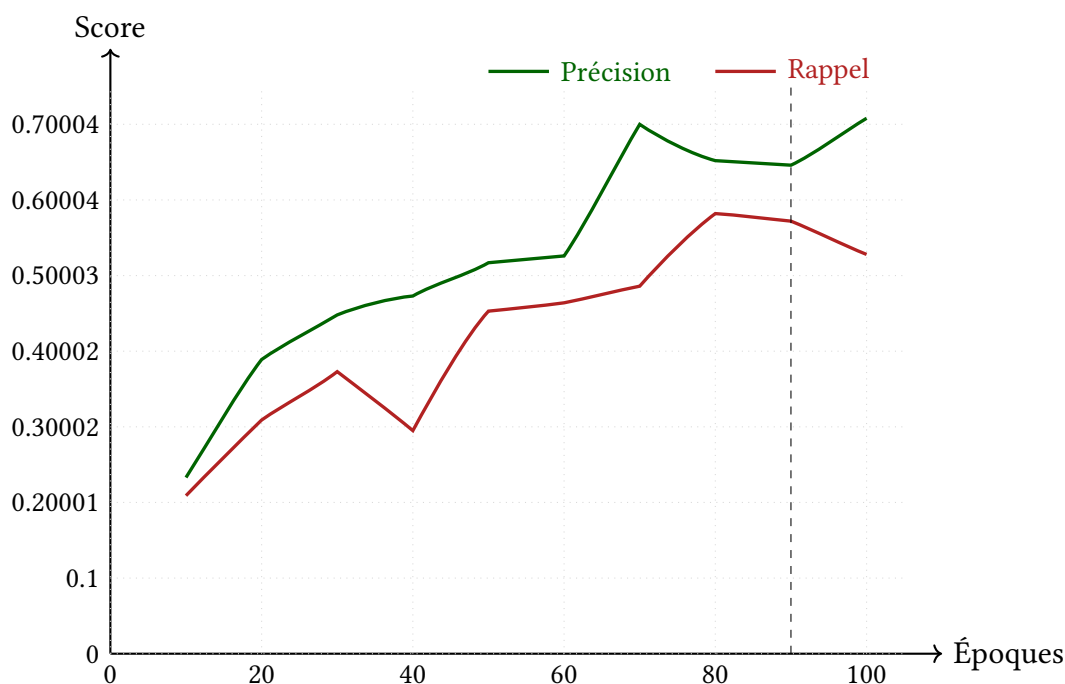


FIGURE 11 – Évolution de la précision et du rappel pendant l'entraînement. La ligne en pointillés indique la désactivation de l'augmentation Mosaic à l'époque 90.

La Figure 12 présente l'évolution des métriques mean Average Precision (mAP). Le mAP50 progresse significativement de 0.127 à 0.661, démontrant une amélioration constante de la capacité du modèle à détecter les objets avec un seuil d'IoU (Intersection over Union) de 0.5. En revanche, le mAP50-95, qui évalue la précision de localisation sur une plage de seuils d'IoU, reste significativement inférieur, atteignant seulement 0.334. Cet écart important (0.661 vs 0.334) révèle une limitation majeure du modèle : s'il détecte correctement l'existence et l'emplacement approximatif des objets, il peine à délimiter avec précision leurs contours exacts.

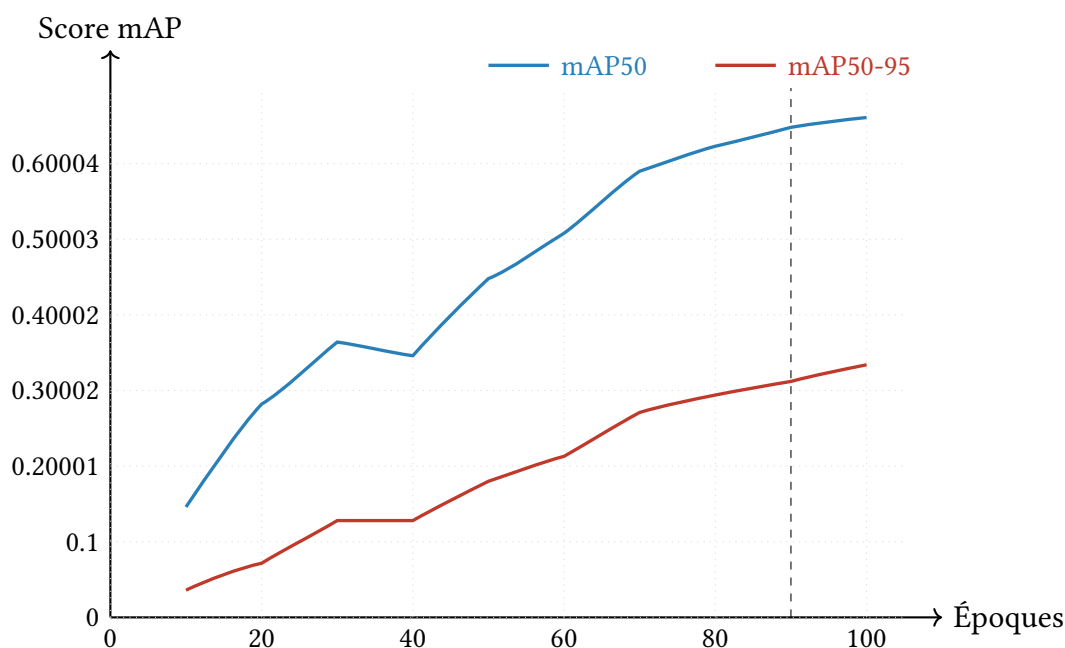


FIGURE 12 – Évolution des métriques mAP pendant l'entraînement. La ligne en pointillés indique la désactivation de l'augmentation Mosaic à l'époque 90.

Le résumé des performances finales présenté dans la figure 13 confirme le profil spécifique de notre modèle : excellente précision (0.708), bon mAP50 (0.661), rappel modéré (0.528) et mAP50-95 limité (0.334). Ce profil est typique des modèles de détection d'objets petits, où la localisation précise représente généralement un défi plus important que la détection de leur présence. Ces résultats s'expliquent par plusieurs facteurs : la difficulté intrinsèque à délimiter précisément de petits objets, les limitations du jeu de données d'entraînement (1800 images), et les particularités de l'architecture YOLOv8 qui, bien qu'efficace pour la détection en temps réel, peut manquer de précision spatiale pour les objets de petite taille. En conclusion, notre modèle spécialisé pour la détection des OP atteint des performances satisfaisantes pour l'identification des objets (excellente précision, bon mAP50), mais présente des limitations concernant l'exhaustivité (rappel modéré) et surtout la précision de localisation (faible mAP50-95). Ces caractéristiques le rendent particulièrement adapté aux applications où la fiabilité des détections prime sur l'exhaustivité et où une localisation approximative est suffisante.

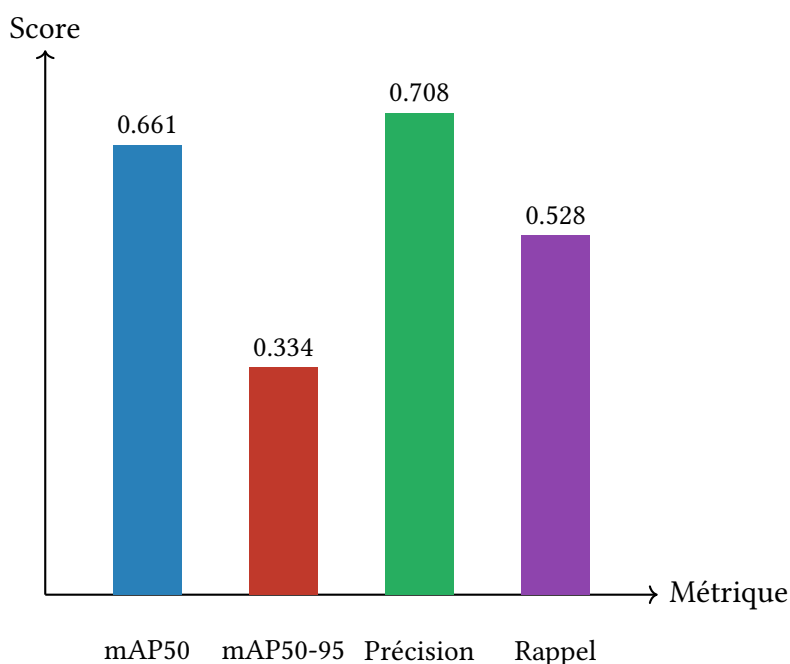


FIGURE 13 – Résumé des performances finales du modèle YOLOv8 spécialisé pour la détection des objets petits (OP).

Hypothèses géométriques pour l'amélioration de la détection des Objets Petits (OP)

Nous avons formulé une proposition théorique d'amélioration pour la détection des OP, bien que cette approche n'ait pas été implémentée dans le cadre actuel de notre étude. Cette piste heuristique repose sur plusieurs considérations géométriques : l'analyse de la position verticale des bounding boxes dans l'image, l'évaluation conjointe de leur taille et position, l'observation de la stabilité temporelle de leurs dimensions, ainsi que l'exploitation potentielle d'informations de profondeur si disponibles.

Le principe fondamental de cette hypothèse repose sur la corrélation entre la position dans l'image et la distance réelle des objets. Plus précisément, nous postulons qu'un objet apparaissant dans la partie inférieure de l'image avec une bounding box de petite dimension correspondrait vraisemblablement à un petit objet situé à proximité. À l'inverse, un objet situé dans la partie supérieure de l'image avec une taille apparente réduite représenterait probablement un objet de taille plus importante mais situé à plus grande distance de la caméra. Cette approche théorique, bien que prometteuse sur le plan conceptuel, nécessiterait des validations expérimentales pour confirmer sa pertinence et son applicabilité dans notre contexte spécifique.

Intégration architecturale L'intégration du modèle spécialisé pour les OP dans le pipeline global a nécessité une modification du système de mapping des classes. Cette adaptation permet un routage intelligent des détections, maintenant le modèle COCO pour les classes standards tout en utilisant le modèle spécialisé pour les OP, avec une correspondance directe entre les catégories OP détectées et leur traitement spécifique.

3.4 Quantification des performances de l'algorithme

Nous avons évalué les performances de notre système de détection sur trois versions successives du pipeline :

1. **Versión initiale** sans tracking, avec les modèles pré-entraînés uniquement.
2. **Versión intermédiaire** intégrant le système de tracking (DeepSort), mais sans fine-tuning.
3. **Versión finale** avec tracking et modèle fine-tuné.

Les métriques utilisées sont la précision, le rappel et le F2-score, ce dernier donnant plus de poids au rappel afin de mieux refléter l'exhaustivité des détections.

Étape	Précision	Rappel	F2-score
Initiale	0.6367	0.5361	0.5743
Intermédiaire (avec tracking)	0.6552	0.7415	0.7329
Finale (avec tracking + fine-tuning)	0.6694	0.8412	0.8001

TABLE 4 – Évolution des métriques au cours des étapes du pipeline

Pour limiter les biais liés à la variabilité du corpus, nous avons effectué les mesures sur des extraits de vidéos variés, en sélectionnant manuellement des séquences contenant différentes scènes, mouvements de caméra, conditions d'éclairage et niveaux de flou. Cela permet de garantir une représentativité réaliste des performances.

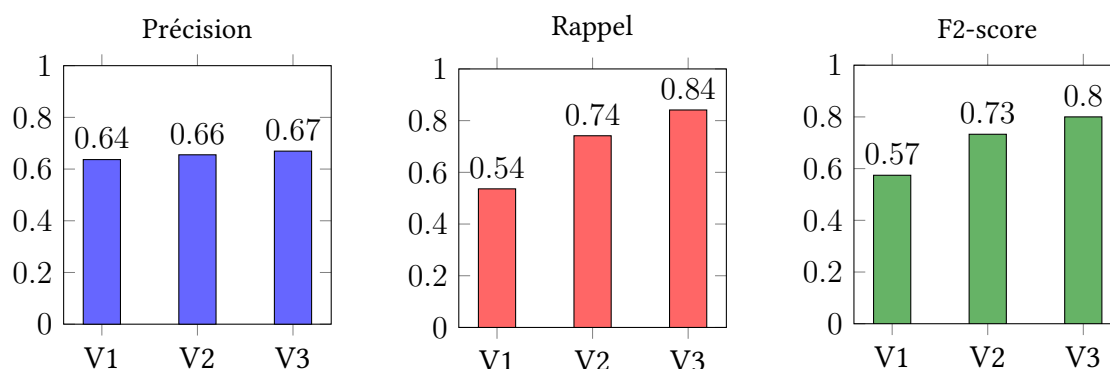


FIGURE 14 – Évolution séparée de la précision, du rappel et du F2-score sur les trois versions du système.

Ces résultats confirment l'importance du tracking pour améliorer le rappel, et du fine-tuning pour stabiliser la précision..

4 Exploitation des résultats

4.1 Choix des vidéos et contraintes techniques

L'objectif initial était de tester notre approche sur des vidéos de deux âges différents d'un même enfant afin d'observer l'évolution des interactions avec les différentes classes au fil du temps. Cependant, lors de la phase de test, nous avons rencontré des dysfonctionnements de la base de données empêchant l'analyse sur l'ensemble des vidéos prévues. Pour pallier ce problème et valider notre méthodologie, nous avons finalement réalisé les courbes à partir d'une seule vidéo test.

L'algorithme de détection génère en sortie un fichier CSV contenant plusieurs informations clés sur les objets détectés. Chaque ligne du fichier correspond à une détection et comprend les éléments suivants :

- Class : la classe de l'objet détecté (ex. : "mains", "visages", "OP", etc.).
- Frame : l'indice de l'image dans la séquence vidéo où l'objet a été détecté.
- ID : un identifiant unique attribué par le tracker pour suivre un même objet sur plusieurs frames.
- Interpolation : indique si la position de l'objet a été calculée par interpolation entre deux détections réelles, afin de pallier les éventuelles absences de détection.
- Coordonnées de la boîte : les coordonnées du rectangle englobant l'objet (bounding box), permettant de situer sa position et sa taille dans l'image.
- Confiance : un score indiquant la fiabilité de la détection, uniquement disponible pour les classes issues d'un modèle pré-entraîné. La classe "OP", en revanche, ne bénéficie pas de ce score de confiance.

```
1 class,frame,track_id,yolo_class,is_interpolated,box,confidence
2 visages,2,,face,False,"[254.9421844482422, -37.683414459228516, 600.0326538085938, 415.2207336425781]",0.9266183972358704
3 visages,65,,face,False,"[123.34578704833984, 55.11881637573242, 219.61302185058594, 169.1400909423828]",0.8627686500549316
4 visages,68,,face,False,"[176.32948303222656, 36.62664031982422, 275.14764404296875, 155.59796142578125]",0.9732796549797058
5 visages,69,,face,False,"[182.03443908691406, 43.87497329711914, 280.8123779296875, 167.37020874023438]",0.9852222800254822
```

FIGURE 15 – Caption

4.2 Choix des indicateurs

La sélection des indicateurs s'est initialement appuyée sur une rencontre avec JRH le 11 mars. L'objectif principal était d'obtenir une distribution spatiale et temporelle des classes. Une autre piste explorée a été l'analyse de la distance entre les mains et les objets, motivée par les recherches sur l'attention des enfants.

L'hypothèse initiale de filtrage des données, qui suggérait la suppression des objets situés en dehors des parties basse et centrale de l'écran, s'est révélée peu pertinente. En effet, les enfants de moins d'un an regardent aussi bien vers le bas que vers le haut, leur attention étant davantage guidée par leur interaction avec l'environnement. De plus, les bébés ont tendance à suivre la direction du regard des adultes, ce qui pourrait être estimé par des modèles de détection du regard. L'inclinaison de la caméra, supposée correspondre à celle du bébé, pourrait également être estimée via l'optical flow.

Les recherches indiquent que les bébés distribuent leur attention visuelle en fonction des tâches et de leur intérêt, plutôt que de se focaliser sur une zone spécifique. La question de l'analyse de la taille et de la vitesse des objets a été soulevée, mais cette approche ne semble pas prometteuse en raison des changements rapides de plans et du manque de représentativité de la taille des objets dans l'image.

Plusieurs études ont éclairé ces choix méthodologiques. L'article de Franchak et al. (2011), *Head-mounted eye tracking : a new method to describe infant looking* [15], montre qu'un système de suivi oculaire monté sur la tête permet de capturer précisément le regard des bébés dans des conditions naturelles entre 8 et 12 mois. Il en ressort que l'attention visuelle des bébés se répartit sur l'ensemble du champ visuel, avec une forte proportion de fixations dirigées vers des objets manipulables et des visages, indépendamment de leur position dans l'espace visuel. Restreindre l'analyse à une zone spécifique risquerait donc d'occulter une part significative des comportements visuels.

Une réflexion similaire a été menée à la lumière de l'étude *Characterizing the object categories two children see and interact with in a dense dataset of naturalistic visual experience* (2021) [16]. Cette étude analyse la présence des différentes catégories d'objets dans le champ visuel des nourrissons et leur évolution au cours du développement. Contrairement à la fréquence d'apparition des mains et des visages, qui évolue avec l'âge, les catégories d'objets restent relativement stables. Les principaux résultats indiquent :

- Une forte prévalence des personnes et des objets, mais une faible présence des animaux.
- Une augmentation de la fréquence des objets manipulables (OP) par rapport aux objets ambiants (OAG) avec l'âge, particulièrement lorsque les mains des enfants sont visibles.
- Des co-occurrences fréquentes entre certaines catégories d'objets, telles que personnes et vêtements, ou animaux, jouets et livres.
- Parmi les OP, les livres sont les plus manipulés, suivis des jouets, avec une distribution conforme à une loi de puissance.

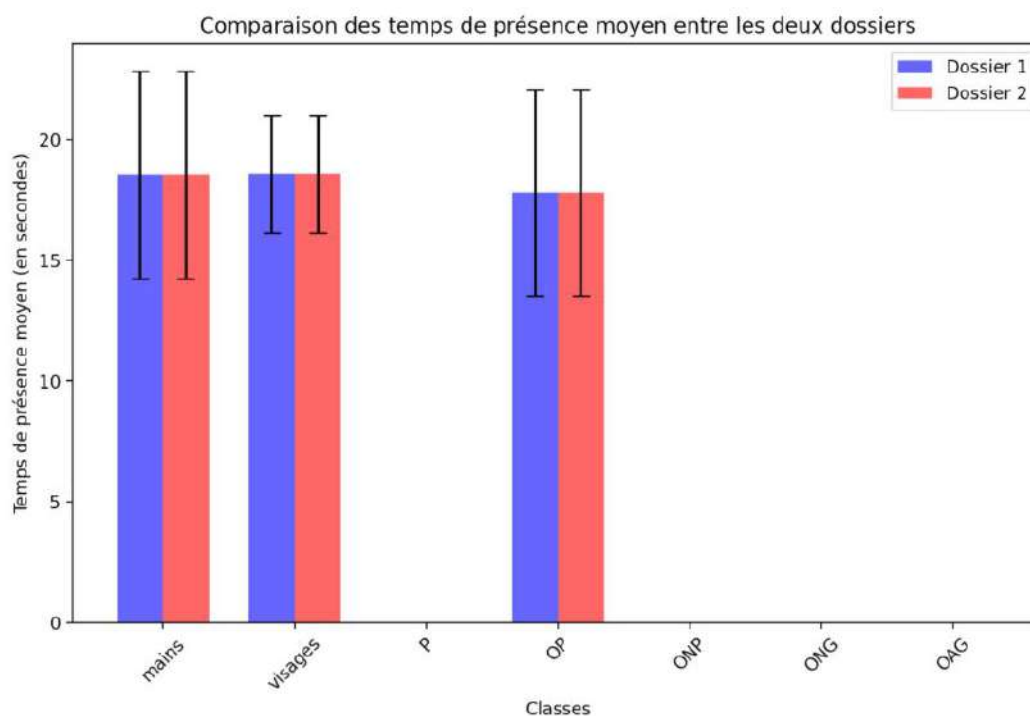
Enfin, l'article *Detecting social information in a dense database of infants' natural visual experience* (2020) [17] du même laboratoire met en évidence l'importance des mains et des visages des aidants dans le développement cognitif et linguistique des nourrissons. L'analyse d'une large base de vidéos egocentriques (SAYCam dataset) a révélé une proportion plus importante de mains dans le champ visuel que précédemment rapporté, ainsi qu'une diminution modérée des visages avec l'âge. Cette étude souligne également l'impact du contexte d'activité sur la présence de ces éléments sociaux dans le champ visuel des enfants.

L'ensemble de ces résultats oriente nos choix d'indicateurs vers une analyse prenant en compte l'analysé des temps de présence des différentes classes, leurs positions dans l'espace et la distance mains-OP.

4.3 Visualisation des résultats

Analyse des Temps de Présence Cet algorithme en Python utilise Streamlit et Pandas pour analyser les temps de présence des différentes classes dans des fichiers CSV. Pandas est une bibliothèque permettant la manipulation et l'analyse efficace des données sous forme de tableaux, tandis que Streamlit est utilisé pour créer une interface web interactive facilitant la visualisation des résultats.

L'algorithme charge et trie les données par classe et par frame, puis calcule la durée de présence et la variance des apparitions. Ensuite, il agrège ces valeurs pour obtenir des moyennes globales. La visualisation se fait via Matplotlib avec un graphique à barres comparant les deux ensembles de données, accompagné d'un tableau interactif affichant les résultats. Grâce à Streamlit, l'application offre une interface permettant d'explorer ces données de manière interactive et efficace.



Analyse des Positions Moyennes En complément de l'analyse temporelle, cet algorithme évalue la répartition spatiale des classes à travers les vidéos annotées. Il extrait les coordonnées des bounding boxes des objets identifiés, puis calcule leurs positions moyennes ainsi que leurs variances pour chaque classe. Ces valeurs permettent d'étudier la répartition et la dispersion des éléments dans l'espace.

Les résultats sont illustrés sous forme d'ellipses tracées sur une image, représentant la moyenne et la variance des positions de chaque classe. La bibliothèque OpenCV est utilisée pour générer ces visualisations, tandis que Matplotlib permet d'afficher les images dans Streamlit. Enfin, un tableau récapitulatif présente les moyennes et les variances des coordonnées X et Y pour chaque classe et chaque jeu de données, facilitant ainsi la comparaison entre différentes conditions expérimentales.

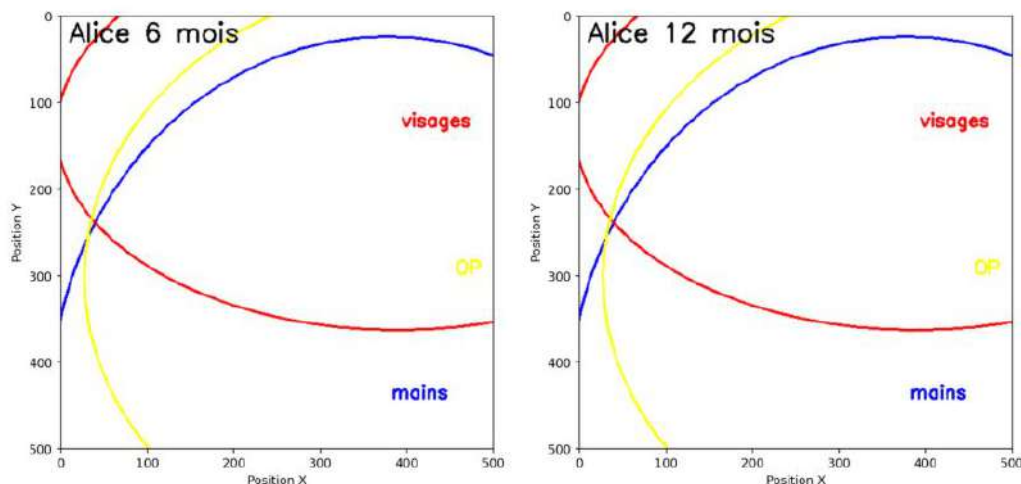
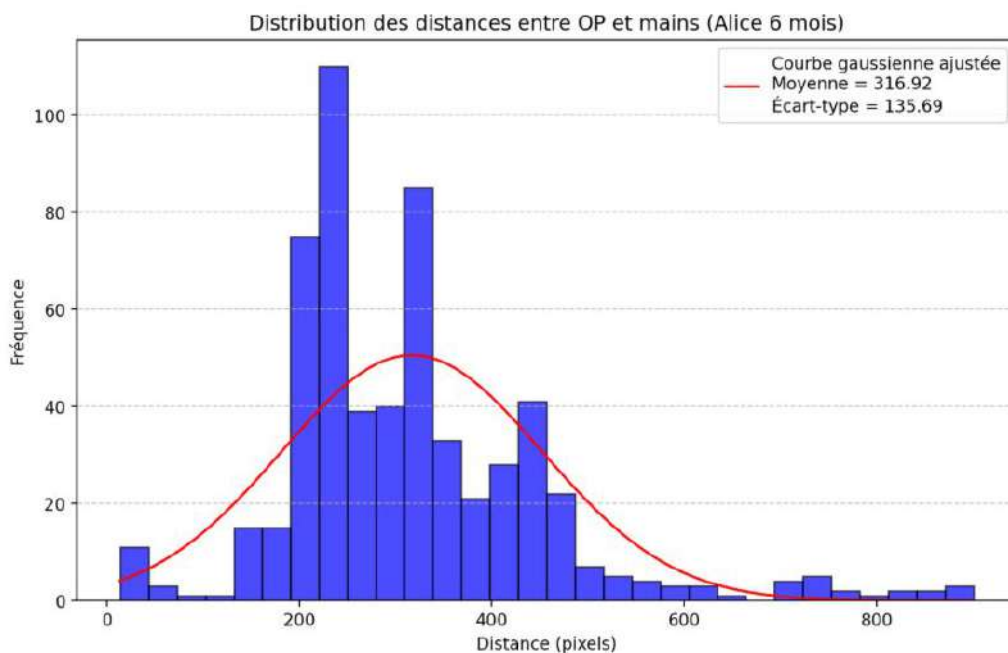


Tableau des moyennes et variances

Classe	Moyenne X (Dossier 1)	Variance X (Dossier 1)	Moyenne Y (Dossier 1)	Variance Y (Dossier 1)	Moyenne X
mains	377.93	16320.41	447.82	12851.73	377.93
visages	389.78	16993.84	133.07	3336.77	389.78

Analyse des Distances entre mains et OP Cet algorithme s'intéresse également aux distances entre objets appartenant à différentes classes. Il extrait les coordonnées des centres des bounding boxes et calcule la distance euclidienne entre chaque paire d'objets d'intérêt pour chaque frame.

Une fois les distances collectées, leur distribution est analysée et visualisée sous forme d'un histogramme, accompagné d'une courbe gaussienne ajustée. Cette représentation permet de mieux comprendre les relations spatiales entre les classes analysées. La bibliothèque SciPy est utilisée pour modéliser cette distribution, tandis que Matplotlib génère les graphiques et Streamlit facilite leur affichage interactif. Cette analyse fournit des informations sur la proximité entre objets et aide à interpréter leur agencement dans l'espace.



5 Pistes d'améliorations et conclusion

5.1 Pistes d'améliorations

Au terme de ce projet, plusieurs axes d'amélioration se dessinent pour renforcer la performance et la robustesse du système de détection et d'analyse vidéo mis en place.

Un premier levier consiste à élargir le jeu de données d'entraînement. Le modèle utilisé repose actuellement sur un corpus relativement restreint (environ 1800 images), ce qui limite ses capacités de généralisation. L'enrichissement de ce jeu de données, en y intégrant une plus grande diversité de contextes, d'éclairages, d'angles de prise de vue et de configurations d'objets, permettrait d'améliorer significativement les performances, notamment en termes de rappel et de précision moyenne (mAP50-95).

Par ailleurs, les hypothèses géométriques formulées dans le cadre théorique – notamment l'analyse de la position verticale des objets, la corrélation taille-position et la stabilité temporelle – mériteraient d'être implémentées et testées. Une telle approche pourrait améliorer la détection des objets petits (OP), en particulier dans les situations ambiguës où les modèles purement visuels peinent à discriminer les catégories.

Une autre piste prometteuse repose sur l'intégration d'informations de profondeur. L'estimation de la profondeur à partir d'images monoculaires via des modèles d'apprentissage profond permettrait de distinguer plus efficacement les petits objets proches des objets volumineux situés en arrière-plan, augmentant ainsi la fiabilité du système.

Enfin, d'autres perspectives pourraient être explorées, telles que l'intégration d'indicateurs comportementaux avancés (temps d'interaction, trajectoires) ou de modules de détection du regard, qui offriraient des clés d'interprétation supplémentaires dans le cadre de l'étude du

développement cognitif.

5.2 Conclusion

Ce projet a permis de développer un pipeline complet de détection et d'analyse d'objets dans des vidéos d'enfants, spécifiquement optimisé pour l'étude du développement cognitif infantile. L'architecture modulaire proposée, combinant un modèle YOLOv8 fine-tuné avec un système de tracking DeepSort, a démontré des performances satisfaisantes, atteignant un F2-score de 0.8001 dans sa version finale.

Les résultats montrent une progression significative à travers les trois versions successives du pipeline, avec une amélioration particulièrement notable du rappel (de 0.5361 à 0.8412) grâce à l'intégration du tracking et du fine-tuning. Cette progression confirme la pertinence de notre approche incrémentale et l'importance cruciale du tracking pour l'analyse de séquences vidéo dynamiques.

Les caractéristiques du modèle spécialisé pour la détection des OP révèlent un profil typique des systèmes de détection d'objets petits : excellente précision (0.708), bon mAP50 (0.661), mais rappel modéré (0.528) et mAP50-95 limité (0.334). Ce profil traduit la difficulté inhérente à localiser précisément de petits objets dans des scènes complexes, mais reste particulièrement adapté aux applications où la fiabilité des détections prime sur leur exhaustivité.

Les indicateurs développés (temps de présence, positions moyennes, distances entre classes) s'inspirent directement des recherches récentes sur l'attention visuelle des nourrissons et offrent des outils pertinents pour l'analyse des interactions enfant-objet. L'interface interactive créée avec Streamlit facilite l'exploration et l'interprétation des résultats, rendant le système accessible aux chercheurs en sciences cognitives.

En conclusion, bien que plusieurs axes d'amélioration restent à explorer, le système développé constitue une base solide pour l'analyse automatisée des comportements d'exploration visuelle et manipulatoire chez les jeunes enfants. Il contribue ainsi à l'étude du développement cognitif en fournissant des métriques objectives et reproductibles sur les interactions enfant-environnement. À terme, ce type d'outil pourrait faciliter la détection précoce de schémas développementaux atypiques et l'évaluation de l'impact des interventions précoces.

6 Dépôt GitHub

Le code source utilisé dans ce projet est disponible sur GitHub à l'adresse suivante :
<https://github.com/encoreunpseudo/parfinal.git>.

Références

Débruitage de Vidéos

- [1] Maggioni, M., Boracchi, G., Foi, A., Egiazarian, K., *Video denoising, deblocking, and enhancement through separable 4-D nonlocal spatiotemporal transforms*, IEEE Transactions

- on Image Processing, vol. 21, no. 9, pp. 3952-3966, 2012.
- [2] Tassano, M., Delon, J., Veit, T., *FastDVDnet : Towards real-time deep video denoising without flow estimation*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 1354-1363, 2020.
 - [3] Cao, K., Ji, Y., Zhang, Z., Zhang, J., Han, Z., *Video Restoration Transformer*, arXiv preprint arXiv :2104.06170, 2021.
 - [4] Dabov, K., Foi, A., Egiazarian, K., *Video denoising by sparse 3D transform-domain collaborative filtering*, European Signal Processing Conference, pp. 145-149, 2007.

Stabilisation de Vidéos

- [5] *Video stabilization : A comprehensive survey*.
- [6] *A Brief Survey on Video Stabilization Approaches*.

Détection d'Objets

- [7] OCTO Technology, *You Only Look Once : Un réseau de neurones pour la détection d'objets*, 2023. Disponible : <https://blog.octo.com/you-only-look-once-un-reseau-de-neurones-pour-la-detection-dobjets>.
- [8] Anil Ak, *Understanding Object Detection with YOLO*, 2020. Disponible : <https://medium.com/@anilaknb/understanding-object-detection-with-yolo-4b658e1efbab>.
- [9] Bochkovskiy, A., Wang, C. Y., & Liao, H. Y. M. (2020). *YOLOv4 : Optimal Speed and Accuracy of Object Detection*. arXiv preprint arXiv :2004.10934.
- [10] Ren, S., He, K., Girshick, R., & Sun, J. (2015). *Faster R-CNN : Towards Real-Time Object Detection with Region Proposal Networks*. In *Advances in Neural Information Processing Systems*, pages 91–99.
- [11] Zhang, K., Zhang, Z., Li, Z., & Qiao, Y. (2016). *Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks*. *IEEE Signal Processing Letters*, 23(10) :1499–1503.
- [12] Deng, J., Guo, J., Zhou, Y., Yu, J., Kotsia, I., & Zafeiriou, S. (2019). *RetinaFace : Single-Stage Dense Face Localisation in the Wild*. arXiv preprint arXiv :1905.00641.
- [13] Bazarevsky, V., Kartynnik, Y., Vakunov, A., Raveendran, K., & Grundmann, M. (2019). *BlazeFace : Sub-millisecond Neural Face Detection on Mobile GPUs*. arXiv preprint arXiv :1907.05047.

Connaissances sur les enfants

- [14] Saycam, *SAYCam : A Large, Longitudinal Audiovisual Dataset Recorded From the Infant's Perspective*.
- [15] Franchak, J. M., Kretch, K. S., Soska, K. C., Adolph, K. E. (2011). Head-mounted eye tracking : a new method to describe infant looking. *Child Development*, 82(6), 1738-1750.

- [16] Long, B., Kachergis, G., Bhatt, N. S., Frank, M. C. (2021). Characterizing the object categories two children see and interact with in a dense dataset of naturalistic visual experience. *Developmental Science*, 24(5), e13023.
- [17] Long, B., Kachergis, G., Agrawal, K., Frank, M. C. (2020). Detecting social information in a dense database of infants' natural visual experience. *Proceedings of the Annual Meeting of the Cognitive Science Society*, 42.